

Redes de Computadores

2024/25

Manual da Disciplina

Versão 4 (2025.03.24)

Pedro Patinho - Departamento de Informática

 **Universidade de Évora**

Este manual é apenas um resumo da matéria, não dispensando a consulta da bibliografia recomendada, nem substituindo a frequência das aulas.

Índice de Aulas

1	Introdução [10.02.2025]	5
1.1	Apresentação da Disciplina	5
1.2	Redes e Internet	6
1.3	Internet e Protocolo IP	7
1.4	Protocolos de Transporte	7
1.5	Camadas de Rede	8
1.6	Aplicações de Rede	11
1.7	Trabalho de Casa	11
1.8	Para lembrar	12
2	Aplicações de Rede e Sockets [17.02.2025]	13
2.1	Introdução	13
2.2	Protocolos de Aplicação e Comunicação End-to-End	13
2.3	Abstração por Sockets	13
2.4	Tipos de Sockets e Particularidades na Programação	14
2.5	Modelo cliente/servidor	14
2.6	Protocolos de Aplicação Comuns	17
2.7	Trabalho de Casa	20
2.8	Conclusão	20
3	Endereçamento IP [24.02.2025]	21
3.1	Introdução	21
3.2	Endereços de Rede	21
3.3	Requisitos para Endereçamento	21
3.4	Estrutura dos Endereços IP	22
3.5	Hierarquia nos Endereços IP	22
3.6	Exemplos Práticos	22
3.7	Recursividade das Redes	22

3.8	Distribuição e Atribuição dos Endereços	22
3.9	Funcionamento do Encaminhamento	24
3.10	Hierarquia e Limitações na Prática	24
3.11	Gamas de Endereços IPv4 Reservados	24
3.12	Conclusão	25
4	O Protocolo IP [10.03.2025]	26
4.1	Introdução e Contextualização	26
4.2	Posição do Protocolo IP na Stack TCP/IP	26
4.3	Formato de um Pacote IP (IPv4)	26
4.4	Encaminhamento de Pacotes: Direto, Indireto e por Omissão	28
4.5	Protocolos Complementares ao IP	30
4.6	Exemplos Práticos e Aplicações	31
4.7	IPv6: A Evolução do Protocolo IP	32
4.8	Network Address Translation (NAT)	33
4.9	Gamas de Endereços IPv4 Privados	34
4.10	Aplicações e Impactos Práticos do Protocolo IP	34
4.11	Desafios e Perspetivas Futuras	34
4.12	Considerações Finais e Reflexões sobre o Protocolo IP	35
4.13	Para Lembrar	35
4.14	Conclusão	36

1. Introdução [10.02.2025]

1.1 Apresentação da Disciplina

O objetivo desta disciplina é introduzir os conceitos fundamentais das redes de computadores, desde a infraestrutura física até às aplicações que utilizam a rede para comunicação. Ao longo do semestre, serão abordados temas como o funcionamento da Internet, os protocolos de comunicação e a organização das redes, bem como a criação de software com comunicação em rede (sockets).

1.1.1 Docente(s)

Docente responsável: Pedro Patinho

- **Email:** pp@uevora.pt
- **Gabinete:** 256, CLV

Aula teórica: segunda@09.00

Turnos práticos: **A** (segunda@14.00), **B** (quinta@14.00) e **C** (quinta@16.00).

1.1.2 Avaliação

A avaliação da disciplina está estruturada em várias componentes, podendo os alunos optar por avaliação contínua ou por exame final.

- **Avaliação contínua** (duas frequências):
 1. 31 de março (60% - realizada durante a aula)
 2. 12 de junho (60% - opcional, os alunos podem optar pelo exame normal)
- **Exame final** (duas chamadas):
 1. 12 de junho (70% - exame normal)
 2. 3 de julho (70% - exame de recurso ou melhoria de nota)
- **Trabalho de Programação em Rede** (30% - obrigatório, nota mínima de 8 valores):

Esta tarefa é obrigatória e independente do formato de avaliação escolhido pelo aluno.

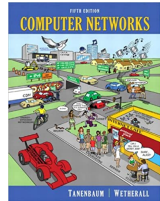
 - Entrega intermédia (a meio do semestre)
 - Entrega final (15 de junho)
 - É obrigatório (i.e., não entrega $\implies$ nota 0)
- **TPC e Trabalhos de Laboratório** (10% - para alunos em avaliação contínua).

1.1.3 Bibliografia Recomendada



Fundamentos de Redes de Computadores

José Legatheaux Martins, Nova.FCT Editorial



Computer Networks

Andrew S. Tanenbaum, Prentice Hall

1.2 Redes e Internet

1.2.1 O que é uma rede?

Uma rede de computadores é um conjunto de dispositivos (computadores) interligados por canais de comunicação (físicos ou virtuais) que permitem a comunicação e a partilha de recursos entre tais computadores. Esses dispositivos podem ser computadores, servidores, impressoras, dispositivos móveis e qualquer outro equipamento que tenha capacidade de comunicação digital.

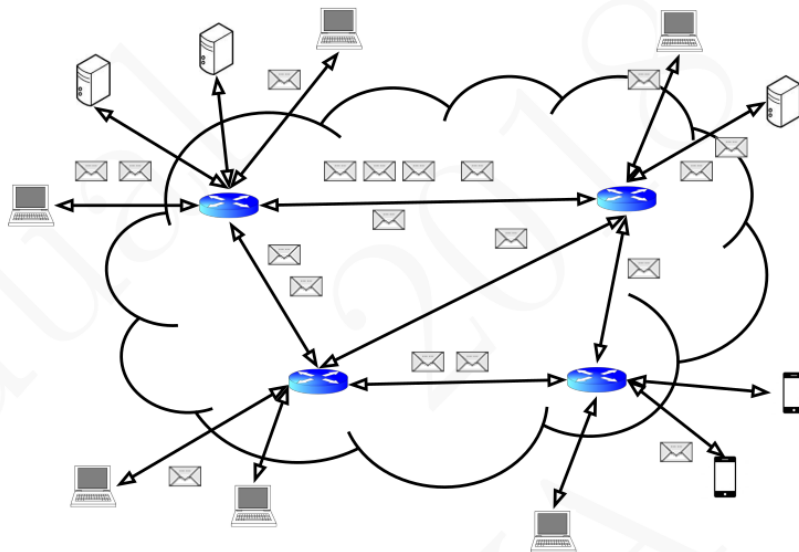


Figura 1.1: Exemplo de rede de computadores

A segmentação dos dados em pacotes permite o envio simultâneo de diferentes fluxos de informação pelo mesmo meio de comunicação, uma técnica conhecida como **multiplexing**.

O tráfego dos dados pela rede e escolha do caminho é gerido por *comutadores (switches)* e *encaminhadores (routers)*.

1.2.2 Protocolos e Camadas de Comunicação

A comunicação em redes divide-se em várias camadas lógicas, que oferecem diversos níveis de abstração sobre os dados em trânsito (e.g., camada física, de rede, de transporte, de aplicação).

Os protocolos definem as regras de comunicação, para cada camada, entre os dispositivos em rede. São normalmente descritos em documentos chamados **RFCs**¹ (*Request For Comments*).

Entre os principais protocolos, destacam-se:

- **IEEE 802.3 (Ethernet)** e **IEEE 802.11 (WiFi)**: definem os standards para interligação em redes locais (camada física).
- **IP (Internet Protocol)**: define os endereços de origem e destino dos pacotes de dados e a “linguagem” base da comunicação intra- e inter-redes (camada de rede).
- **TCP (Transmission Control Protocol)** e **UDP (User Datagram Protocol)**: regulam a transmissão de dados entre aplicações (camada de transporte).
- **HTTP (HyperText Transfer Protocol)**: define a “linguagem” para comunicação entre servidores e clientes WWW (*World Wide Web*) (camada de aplicação).
- **DNS (Domain Name System)**: define a “linguagem” para perguntas e respostas sobre os nomes de domínio e correspondentes endereços IP (camada de aplicação) - a chamada “lista telefónica” da Internet.

1.3 Internet e Protocolo IP

A Internet é uma **rede de redes**, interligando milhões de dispositivos globalmente. Os seus principais princípios incluem:

- Diversidade de tecnologias e infraestruturas
- Encaminhamento eficiente de pacotes
- Funcionamento em **best-effort**, ou seja, sem garantias absolutas de entrega

Cada dispositivo conectado à Internet possui um (ou mais) endereço(s) **IP**, que é um número de 32bit²:

- Exemplo em decimal: 3246970901
- Exemplo em binário: 11000001100010001101100000010101
- Equivalente em notação *dotted decimal*: 193.136.216.21

Na prática, é atribuído um endereço IP a cada *interface de rede* presente no computador.

A figura 1.2 mostra o formato de um pacote IP.

1.4 Protocolos de Transporte

1.4.1 TCP (Transmission Control Protocol)

O TCP é o protocolo de transporte mais utilizado em redes IP, e caracteriza-se pelo seguinte:

- Garante a ordem dos pacotes
- Garante a entrega, através de retransmissão
- Mantém uma conexão *aberta* entre as duas partes, usando **portas** específicas de origem e destino para cada conexão
- Permite adaptar a velocidade de transmissão, de acordo com o grau de *saturação* da rede

A figura 1.3 mostra o formato de um pacote TCP, incluindo o cabeçalho IP visto anteriormente (figura 1.2).

1.4.2 UDP (User Datagram Protocol)

O UDP, não oferecendo as garantias do TCP, é um protocolo mais “leve” (apenas 8 bytes de cabeçalho).

- Baseia-se em “datagramas”

¹<https://www.ietf.org/standards/rfcs/>

²O IPv4, mais generalizado pela Internet, usa endereços de 32bit. O IPv6, mais moderno mas ainda não usado em todas as redes, usa endereços de 128bit. Entraremos em maior detalhe mais à frente.

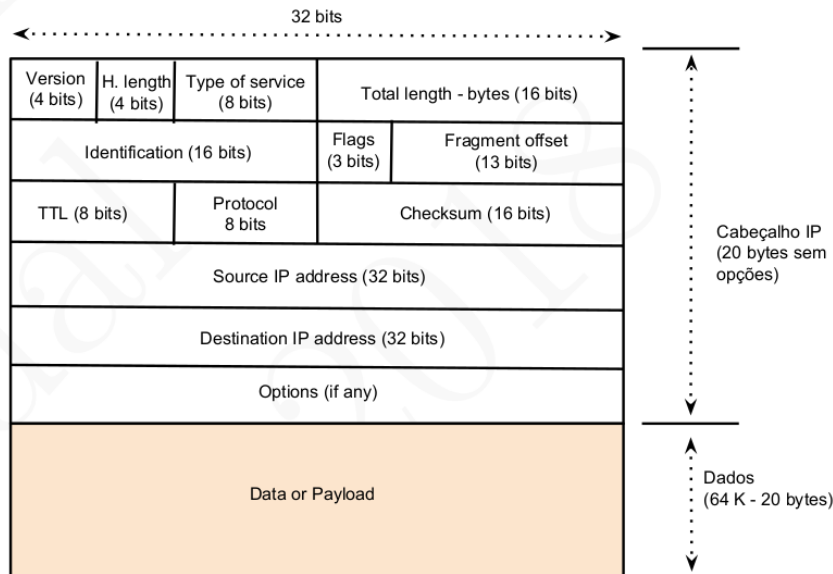


Figura 1.2: Formato de um pacote IP

- *Connectionless* (não estabelece conexão antes da transmissão)
- Não garante ordem nem retransmissão de pacotes
- Ideal para aplicações como streaming de vídeo e jogos online

A figura 1.4 mostra o formato de um pacote TCP, incluindo o cabeçalho IP visto anteriormente (figura 1.2).

1.5 Camadas de Rede

As redes de computadores são organizadas em camadas, cada uma com funções específicas, facilitando abstração sobre a comunicação entre dispositivos heterogêneos a diferentes níveis. A separação em camadas permite modularidade e interoperabilidade e facilita o desenvolvimento independente de protocolos para cada camada.

1.5.1 Modelo OSI (Open Systems Interconnection)

O modelo OSI, desenvolvido pela *International Organization for Standardization* (ISO) em 1983, define uma arquitetura de redes baseada em sete camadas:

1. **Física:** Define as características elétricas, mecânicas e funcionais para a transmissão de bits através dos meios físicos.
2. **Ligação de Dados:** Garante a comunicação sem erros entre dispositivos adjacentes, organizando os bits em **frames** e controlando o acesso ao canal físico.
3. **Rede:** Responsável pelo endereçamento e encaminhamento de pacotes entre redes distintas (exemplo: protocolo IP).
4. **Transporte:** Garante a entrega fiável e ordenada dos dados entre dispositivos finais (exemplos: TCP e UDP).
5. **Sessão:** Controla o estabelecimento, manutenção e encerramento de conexões entre aplicações.
6. **Apresentação:** Tradução, compressão e criptografia dos dados para garantir que diferentes sistemas possam comunicar corretamente entre si.
7. **Aplicação:** Fornece serviços directamente às aplicações de uso "final", como browsers HTTP, FTP,

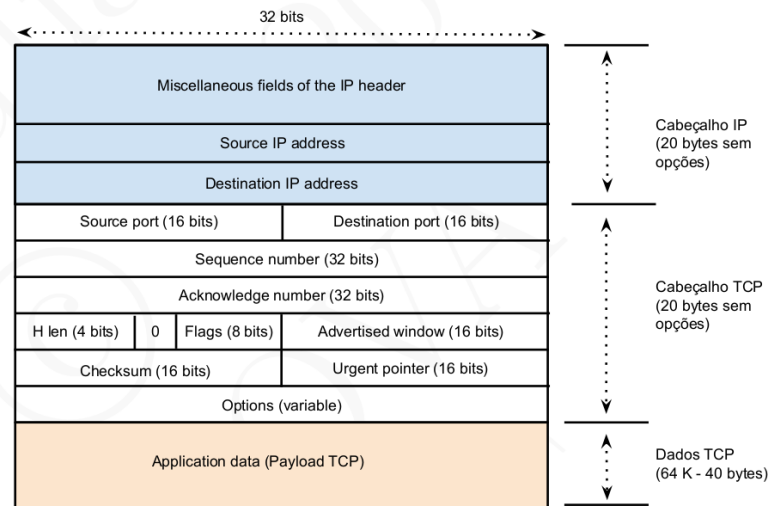


Figura 1.3: Formato de um pacote TCP

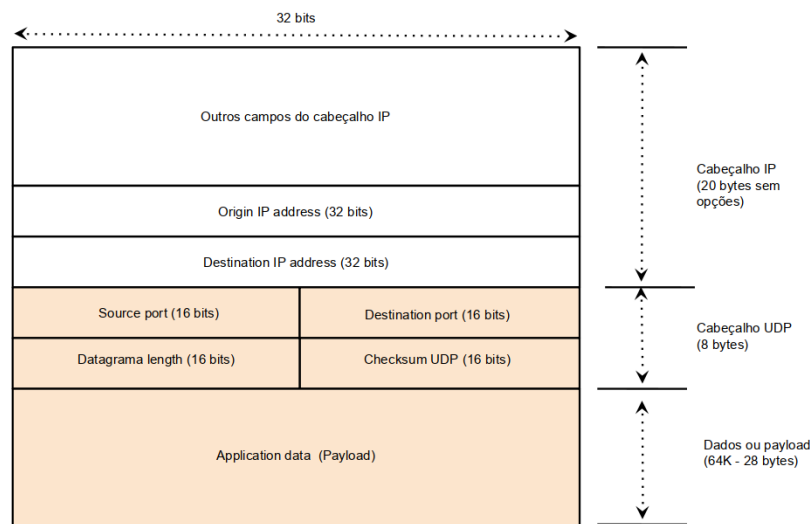


Figura 1.4: Formato de um pacote UDP

clientes de e-mail, entre outros.

A Figura 1.5 ilustra a estrutura do modelo OSI e as suas camadas.

Apesar de ser um modelo teórico útil, o modelo OSI não foi amplamente adotado na prática. Em vez disso, a **stack TCP/IP** tornou-se o padrão predominante para redes modernas, devido à sua escalabilidade, modularidade e facilidade de implementação e aperfeiçoamento dos protocolos.

1.5.2 Modelo TCP/IP

O modelo TCP/IP, desenvolvido pelo Departamento de Defesa dos EUA na década de 70, simplificou a arquitetura OSI e tornou-se a base para a criação da Internet. É composto por **quatro** camadas principais:

1. **Ligação:** Equivalente às camadas Física e de Ligação de Dados do modelo OSI. Define a transmissão dos dados pelo meio físico e o controlo de acesso à rede.
2. **Rede:** Responsável pelo endereçamento e encaminhamento dos pacotes de dados, utilizando o protocolo IP.
3. **Transporte:** Garante a comunicação *end-to-end* entre aplicações, através de protocolos como TCP

	Number	Name	Description/Example
Hosts	7	Application	Specifies methods for accomplishing some user-initiated task. Application-layer protocols tend to be devised and implemented by application developers. Examples include FTP, Skype, etc.
	6	Presentation	Specifies methods for expressing data formats and translation rules for applications. A standard example would be conversion of EBCDIC to ASCII coding for characters (but of little concern today). Encryption is sometimes associated with this layer but can also be found at other layers.
	5	Session	Specifies methods for multiple connections constituting a communication session. These may include closing connections, restarting connections, and checkpointing progress. ISO X.225 is a session-layer protocol.
	4	Transport	Specifies methods for connections or associations between multiple programs running on the same computer system. This layer may also implement reliable delivery if not implemented elsewhere (e.g., Internet TCP, ISO TP4).
All Networked Devices	3	Network or Internetwork	Specifies methods for communicating in a multihop fashion across potentially different types of link networks. For packet networks, describes an abstract packet format and its standard addressing structure (e.g., IP datagram, X.25 PLP, ISO CLNP).
	2	Link	Specifies methods for communication across a single link, including "media access" control protocols when multiple systems share the same media. Error detection is commonly included at this layer, along with link-layer address formats (e.g., Ethernet, Wi-Fi, ISO 13239/HDLC).
	1	Physical	Specifies connectors, data rates, and how bits are encoded on some media. Also describes low-level error detection and correction, plus frequency assignments. We mostly stay clear of this layer in this text. Examples include V.92, Ethernet 1000BASE-T, SONET/SDH.

Figura 1.5: Modelo OSI

(fiável) e UDP (não fiável).

4. **Aplicação:** Equivalente às camadas de Sessão, Apresentação e Aplicação do modelo OSI. Contém os protocolos de comunicação usados pelos programas de utilizador, como HTTP(s), (s)FTP, SMTP, POP, IMAP, etc.

A Figura 1.6 ilustra a estrutura do modelo TCP/IP e suas camadas.

	Number	Name	Description / Example	
Hosts	7	Application	Virtually any Internet-compatible application, including the Web (HTTP), DNS (Chapter 11), DHCP (Chapter 6).	
	4	Transport	Provides exchange of data between abstract "ports" managed by applications. May include error and flow control. Examples: TCP (Chapters 13-17), UDP (Chapter 10), SCTP, DCCP.	
All Internet Devices	3.5	Network (Adjunct)	Unofficial "layer" that helps accomplish setup, management, and security for the network layer. Examples: ICMP (Chapter 8) and IGMP (Chapter 9), IPsec (Chapter 18).	"Network Layer"
	3	Network	Defines abstract datagrams and provides routing. Examples include IP (32-bit addresses, 64KB maximum size) and IPv6 (128-bit addresses, up to 4GB maximum size). Chapters 2,5.	
	2.5	Link (Adjunct)	Unofficial "layer" used to map addresses used at the network to those used at the link layer on multi-access link-layer networks. Example: ARP (Chapter 4).	"Driver"

Figura 1.6: Modelo TCP/IP e suas quatro camadas

1.5.3 Comparação entre os Modelos OSI e TCP/IP

O modelo OSI é mais detalhado, enquanto o modelo TCP/IP é mais pragmático e utilizado na prática. Algumas diferenças notáveis:

- O modelo OSI tem sete camadas, enquanto o TCP/IP tem quatro.
- O modelo OSI separa as funções de Sessão e Apresentação, enquanto no TCP/IP essas funções são absorvidas na camada de Aplicação.

- O modelo TCP/IP foi criado para a Internet e baseia-se na comunicação entre redes heterogêneas, enquanto o OSI surgiu como um modelo teórico.

Ambos os modelos servem para estruturar o estudo das redes de computadores, sendo que o TCP/IP é o efetivamente utilizado na Internet e na maior parte das redes modernas.

1.6 Aplicações de Rede

Aplicações de rede são programas que utilizam a infraestrutura de comunicação para fornecer serviços diversos. Estes serviços podem incluir navegação na web, correio eletrónico, transferência de ficheiros e comunicação em tempo real, entre outros.

Podemos definir uma aplicação de rede como um processo que é executado num ou mais *hosts*, obedecendo a um protocolo de camada de aplicação e utilizando os serviços das camadas inferiores (e.g., transporte, rede).

Uma aplicação de rede pode ser classificada com base na sua arquitetura e no protocolo de comunicação utilizado. As duas arquiteturas mais comuns são:

- **Modelo Cliente/Servidor**
- **Modelo Peer-to-Peer (P2P)**

1.6.1 Modelo Cliente/Servidor

O modelo **Cliente/Servidor** é uma abordagem tradicional em que:

- Um **cliente** solicita serviços ou dados, usando um sistema de *request/reply*.
- Um **servidor** recebe os pedidos e responde adequadamente.

Os servidores são normalmente sistemas robustos e sempre disponíveis (i.e., correm *para sempre*), enquanto que os clientes são programas que são lançados por iniciativa (e necessidade) do utilizador.

Neste modelo, o servidor aguarda ligações na rede numa porta específica. Quando um cliente estabelece ligação, ocorre a troca de informações baseada no protocolo de aplicação.

Os servidores podem ser classificados em dois tipos:

- **Iterativo:** Processa um pedido de cada vez, esperando que o anterior termine antes de aceitar um novo.
- **Concorrente:** Processa múltiplos pedidos em simultâneo, utilizando técnicas como multiprocessamento ou multithreading.

1.6.2 Modelo Peer-to-Peer (P2P)

O modelo **Peer-to-Peer (P2P)** elimina a distinção entre clientes e servidores. Cada dispositivo na rede pode tanto requisitar serviços como oferecê-los. Isto resulta numa arquitetura descentralizada e escalável.

Em aplicações P2P, cada processo em execução é um *peer* e pode partilhar recursos sem a necessidade de um servidor central. O protocolo BitTorrent é um exemplo deste modelo:

- Um peer pode solicitar bocados de um ficheiro a outros peers.
- Ao mesmo tempo, pode fornecer bocados já adquiridos a outros peers.
- Peers que já têm o ficheiro completo são chamados **seeds**.

1.7 Trabalho de Casa

Os alunos devem atualizar o glossário no Moodle e introduzir os seguintes termos:

- Host
- Protocolo
- Pacote

- Porta
- Cliente
- Servidor
- Peer
- Socket

O uso de ferramentas de I.A. (e.g., ChatGPT e similares) é permitido, desde que os alunos não façam *copy+paste* do resultado (i.e., **têm** de usar as suas próprias palavras para descrever os termos em questão e verificar a credibilidade dos resultados).

1.8 Para lembrar

Esta secção apresenta um resumo dos conceitos fundamentais apresentados na introdução à disciplina de Redes de Computadores.

1.8.1 Conceitos Fundamentais

- **Rede de Computadores:** Sistema interligado que permite a comunicação e/ou partilha de recursos. Compreende canais físicos e virtuais, encaminhadores e computadores.
- **Protocolo de Comunicação:** Conjunto de regras que definem o formato da informação trocada pelos dispositivos.
- **Pacote de Dados:** Pequena unidade de informação transmitidas na rede, obedecendo a um formato definido por um protocolo (ex: pacote TCP, pacote IP, ...).
- **Modelos OSI e TCP/IP:** Modelos em camadas encapsuladas que organizam a comunicação em redes.

1.8.2 Considerações sobre a Internet e o protocolo IP

A Internet é uma infraestrutura descentralizada baseada no envio de pacotes por redes heterogéneas. Entre as características mais importantes, podemos destacar as seguintes:

- Comunicação por **best-effort**, sem garantia de entrega ou ordem dos pacotes.
- Utilização de **endereços IP** para identificar dispositivos e **DNS** para facilitar a navegação.
- Adopção de protocolos padronizados (**RFCs**) para garantir interoperabilidade.

1.8.3 Conclusão

Nesta aula introdutória foram discutidos os princípios básicos das redes de computadores, incluindo modelos, protocolos, endereçamento e funcionamento básico de aplicações de rede. Embora se tenha dado apenas uma visão geral dos conceitos, o objectivo é criar uma fundação que invoque curiosidade para os próximos passos de aprofundamento destes conceitos.

2. Aplicações de Rede e *Sockets* [17.02.2025]

2.1 Introdução

O principal propósito de uma rede de computadores é possibilitar o funcionamento de aplicações de rede. Se não existissem aplicações úteis a funcionar numa rede, não haveria razão para a rede sequer existir! Embora existam aplicações de rede desde que as redes existem, a partir da criação da Internet foram desenvolvidas inúmeras aplicações com utilidades diversas, desde a gestão e manutenção das próprias redes até ao puro entretenimento. Estas aplicações têm sido o motor para o sucesso da Internet, chegando aos dias de hoje onde esta faz parte integrante de uma boa parte do dia-a-dia de cada um de nós.

Nesta aula, abordamos o funcionamento das aplicações de rede, focando-nos no modelo cliente/servidor e, em particular, na abstração oferecida pelos *sockets* para a comunicação entre processos. Embora ainda não nos aprofundemos nos detalhes de baixo nível dos protocolos IP, TCP ou UDP, esta introdução fornecerá a base necessária para os alunos começarem a implementar aplicações simples de rede.

2.2 Protocolos de Aplicação e Comunicação End-to-End

As aplicações de rede comunicam através de protocolos de aplicação, que definem a estrutura e o formato das mensagens trocadas entre clientes e servidores. Alguns exemplos comuns incluem HTTP, FTP, SMTP e DNS. Estes protocolos funcionam na camada de aplicação do modelo TCP/IP e assumem que a comunicação entre os processos pode ocorrer de forma fiável (TCP) ou *connectionless* (UDP).

O termo **end-to-end** refere-se ao facto de a comunicação ocorrer diretamente entre processos de aplicação em diferentes computadores, independentemente das redes intermediárias e dos processos de baixo nível (que ocorrem “por baixo do capot”). A infraestrutura subjacente trata do encaminhamento e entrega dos pacotes, enquanto que os protocolos de aplicação garantem que a mensagem recebida pelo destinatário cumpre com as normas estabelecidas na “linguagem” utilizada (i.e., obedecem ao **protocolo**).

2.3 Abstração por Sockets

Os *sockets* são uma abstração fornecida pelos sistemas operativos para permitir a comunicação entre processos de rede. Podem ser vistos como pontos de extremidade (*endpoints*) de uma comunicação, permitindo a troca de dados entre computadores. Assim, os *sockets* funcionam idealmente em pares ligados entre si¹.

¹Embora possamos ter *sockets* não ligados (*disconnected sockets*) ou à espera de ligação (no caso de servidores). Nestes casos, os *sockets* não terão funcionalidade prática.

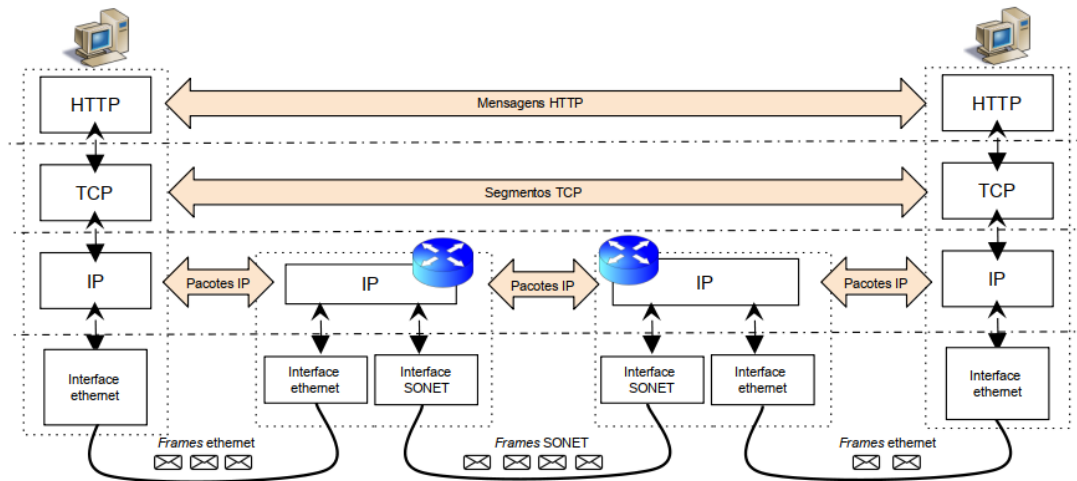


Figura 2.1: Princípio end-to-end.

Em termos práticos, um *socket* (ligado) é como um ficheiro, podendo ser alvo de operações de leitura e escrita, com a particularidade de que tudo o que é escrito (`write()` ou `send()`) num socket pode ser lido (`read()` ou `recv()`) no socket correspondente ao outro endpoint, e vice-versa.

Um socket é identificado por três elementos principais:

- O endereço IP da interface de rede
- O número da porta
- O protocolo de transporte (TCP, UDP, ...)

A API de sockets, originalmente desenvolvida para o sistema UNIX, foi adotada por vários sistemas operativos e é a base da programação de redes em praticamente todas as linguagens de programação atuais.

2.4 Tipos de Sockets e Particularidades na Programação

Existem diferentes tipos de sockets, cada um adequado a diferentes necessidades de comunicação, sendo os mais comuns:

- **Sockets TCP:** Garantem uma comunicação fiável e orientada à ligação.
- **Sockets UDP:** Baseiam-se em datagramas, sem garantia de entrega.
- **Sockets RAW:** Permitem a interação direta com protocolos de rede de baixo nível.

Além da escolha do tipo de socket, na programação de aplicações de rede é necessário considerar o comportamento das operações de I/O:

- **Blocking Sockets:** Operações como `recv()` e `accept()` bloqueiam a execução até que haja dados disponíveis.
- **Non-Blocking Sockets:** Permitem que um processo continue a execução enquanto aguarda comunicação.
- **Multiplexing:** Técnicas como `select()` ou `poll()` permitem gerir múltiplos sockets simultaneamente.

2.5 Modelo cliente/servidor

No modelo cliente/servidor, o servidor abre um socket associado a uma (ou todas) interface de rede e a uma porta específica. Quando um socket cliente se liga àquela interface na porta especificada, o servidor fica com

um segundo socket por onde se fará a comunicação com o socket do cliente.

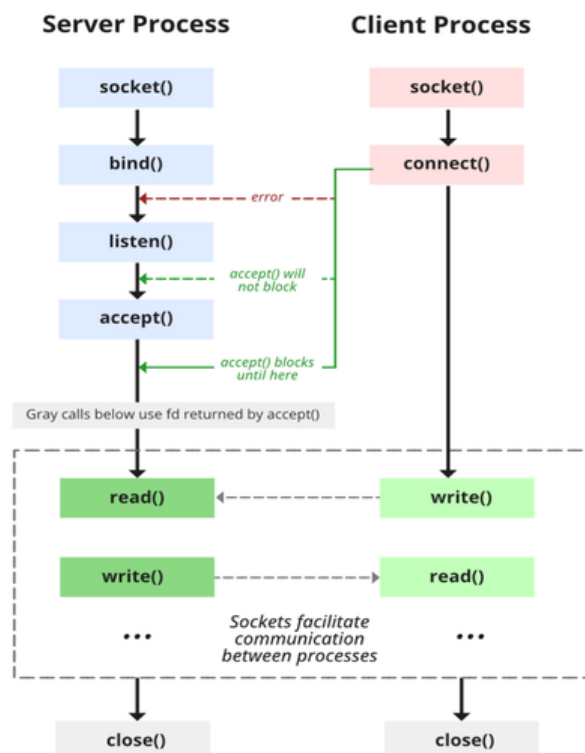


Figura 2.2: Funcionamento programático do modelo cliente/servidor com sockets.

2.5.1 Funcionamento de um Servidor - Exemplo em C

A seguir, um exemplo de implementação de um servidor TCP simples em C:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define BUFFER_SIZE 1024
9
10 int main() {
11     int server_fd, new_socket;
12     struct sockaddr_in address;
13     int addrlen = sizeof(address);
14     char buffer[BUFFER_SIZE] = {0};
15
16     // Criar socket
17     if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
18         perror("Erro ao criar socket");
19         exit(EXIT_FAILURE);
20     }
21
22     address.sin_family = AF_INET;
23     address.sin_addr.s_addr = INADDR_ANY;
  
```

```

24     address.sin_port = htons(PORT);
25
26     // Associar socket a um endereço e porta
27     if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
28         perror("Erro no bind");
29         exit(EXIT_FAILURE);
30     }
31
32     // Colocar socket em modo de escuta
33     if (listen(server_fd, 3) < 0) {
34         perror("Erro no listen");
35         exit(EXIT_FAILURE);
36     }
37
38     printf("Servidor à espera de conexões na porta %d...\n", PORT);
39
40     while (1) {
41         if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
42                                 (socklen_t *)&addrlen)) < 0) {
43             perror("Erro ao aceitar conexão");
44             exit(EXIT_FAILURE);
45         }
46         read(new_socket, buffer, BUFFER_SIZE);
47         printf("Mensagem recebida: %s\n", buffer);
48         send(new_socket, buffer, strlen(buffer), 0);
49         close(new_socket);
50     }
51     return 0;
52 }

```

2.5.2 Funcionamento de um Cliente - Exemplo em C

Agora, um exemplo de implementação de um cliente TCP simples em C:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6
7 #define PORT 8080
8 #define BUFFER_SIZE 1024
9
10 int main() {
11     int sock;
12     struct sockaddr_in server_addr;
13     char buffer[BUFFER_SIZE] = "Olá, servidor!";
14
15     // Criar o socket
16     if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
17         perror("Erro ao criar socket");
18         exit(EXIT_FAILURE);
19     }
20
21     server_addr.sin_family = AF_INET;
22     server_addr.sin_port = htons(PORT);

```



```

23     server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
24
25     // Conectar ao servidor
26     if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
27         perror("Erro ao conectar");
28         exit(EXIT_FAILURE);
29     }
30
31     send(sock, buffer, strlen(buffer), 0);
32     read(sock, buffer, BUFFER_SIZE);
33     printf("Resposta do servidor: %s\n", buffer);
34
35     close(sock);
36     return 0;
37 }

```

2.6 Protocolos de Aplicação Comuns

Os protocolos da camada de aplicação são responsáveis pela interação direta entre o software (e, por inerência, os utilizadores) e os serviços de rede. Nesta secção, analisamos alguns dos protocolos mais comuns, explorando alguns exemplos de mensagens trocadas obedecendo a esses protocolos.

2.6.1 Ligações fiáveis e não fiáveis

As aplicações de rede podem necessitar de maior ou menor fiabilidade, consoante a *elasticidade* e a *tolerância a perdas*. Na tabela 2.1 podemos ver exemplos de aplicações e a sua elasticidade e tolerância a falhas.

Aplicação	Tolerância	Transmissão	Time-sensitive
Download de ficheiros	Sem perdas	Elástica	Não
E-mail	Sem perdas	Elástica	Não
Páginas web	Sem perdas	Elástica	Não
Chamadas de áudio e/ou vídeo	Tolerante a perdas	1kbps - 5Mbps	Sim (milissegundos)
Streaming de vídeo	Tolerante a perdas	~1 - 5Mbps	Sim (poucos segundos)
Jogos	Alguma tolerância	~10kbps	Sim (milissegundos)

Tabela 2.1: Elasticidade e tolerância a falhas.

2.6.2 HTTP - Hypertext Transfer Protocol

O protocolo HTTP (Hypertext Transfer Protocol) é utilizado para a transferência de páginas web e recursos relacionados. Funciona sobre TCP na porta 80 (por defeito) e define a comunicação entre clientes (*browsers*) e servidores web (e.g., Apache, nginx, ...). A versão mais amplamente adotada é o HTTP versão 1.1, definido na RFC 2616². A figura 2.3 mostra o formato de um pedido HTTP.

Exemplo de pedido HTTP:

```

GET /index.html HTTP/1.1
Host: www.exemplo.com
User-Agent: Mozilla/5.0

```

²<https://www.rfc-editor.org/rfc/rfc2616>

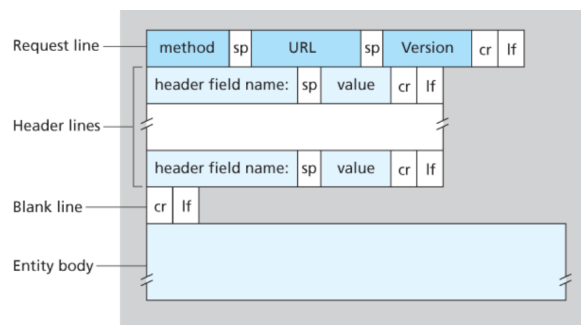


Figura 2.3: Pedido HTTP.

Exemplo de resposta HTTP:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 125

<html>
<head><title>Exemplo</title></head>
<body><h1>Bem-vindo</h1></body>
</html>
```

2.6.3 FTP - File Transfer Protocol

O protocolo FTP (File Transfer Protocol) é utilizado para transferência de ficheiros entre clientes e servidores, em ambos os sentidos. Utiliza duas ligações: uma para controlo (porta 21) e outra para a transferência dos dados. O protocolo FTP está definido na RFC 959³.

Exemplo de sessão FTP:

```
<< 220 FTP Server Ready
>> USER utilizador
<< 331 Password required
>> PASS password
<< 230 User logged in
>> LIST
<< 150 Opening ASCII mode data connection for file list
<< 226 Transfer complete
>> QUIT
<< 221 Goodbye
```

2.6.4 SMTP - Simple Mail Transfer Protocol

O SMTP (Simple Mail Transfer Protocol) é o protocolo padrão para envio de emails. Funciona sobre TCP na porta 25 e está definido na RFC 5321⁴.

³<https://www.rfc-editor.org/rfc/rfc959>

⁴<https://www.rfc-editor.org/rfc/rfc5321>

Exemplo de sessão SMTP:

```
<< 220 mail.example.com ESMTP Postfix
>> HELO cliente.exemplo.com
<< 250 mail.example.com
>> MAIL FROM:<remetente@exemplo.com>
<< 250 OK
>> RCPT TO:<destinatario@exemplo.com>
<< 250 OK
>> DATA
<< 354 End data with <CRLF>.<CRLF>
>> Subject: Teste
>>
>> Corpo do email.
>> .
<< 250 Message accepted
>> QUIT
<< 221 Bye
```

2.6.5 DNS - Domain Name System

O DNS (Domain Name System) é um sistema hierárquico para a resolução de nomes de domínio em endereços IP. Opera geralmente sobre UDP na porta 53 e está definido na RFC 1035⁵.

Exemplo de consulta DNS (adaptada para melhor compreensão):

```
;; Querying for A record of www.exemplo.com
;; Client sends:

www.exemplo.com. IN A

;; Server responds:

www.exemplo.com. 86400 IN A 192.168.1.1
```

O DNS é um protocolo que funciona com uma hierarquia de servidores, responsáveis pela gestão dos diferentes níveis de domínios da Internet:

- **Root servers** - Os servidores "principais" do sistema DNS;
- **TLD servers** - Os servidores de *Top Level Domains* são responsáveis pelos domínios de topo (e.g., .pt, .br, .es, .uk, .com);
- **Authoritative servers** - Responsáveis pelos diversos domínios da Internet (e.g., uevora.pt);
- **ISP (local) servers** - Responsáveis por nos transmitir a informação de domínios ao nosso computador pessoal, são geralmente *não autoritativos*.

⁵<https://www.rfc-editor.org/rfc/rfc1035>

2.7 Trabalho de Casa

Faça uma breve pesquisa e responda à seguinte questão:

Como funciona a distribuição de vídeo da Netflix?

(Em particular, veja como funciona o protocolo de streaming **DASH** e a CDN da Netflix baseada na Cloud da Amazon)

2.8 Conclusão

Os protocolos de aplicação desempenham um papel essencial na comunicação em redes, permitindo a interação direta entre utilizadores e serviços. Nesta secção explorámos os protocolos HTTP, FTP, SMTP e DNS, com exemplos dos tipos de mensagens que possibilitam trocar. Ter uma noção do funcionamento destes e de outros protocolos é fundamental para o desenvolvimento de boas aplicações de rede.

Os sockets são a base da programação de redes, permitindo a comunicação entre aplicações em diferentes computadores. Compreender os tipos de sockets e os seus modos de funcionamento é essencial para desenvolver aplicações de rede eficientes.

Esta introdução fornece um primeiro contacto com a programação de redes e estabelece as bases para tópicos mais avançados, como o controlo de ligações, multiplexação de sockets e segurança das comunicações.

3. Endereçamento IP [24.02.2025]

3.1 Introdução

A comunicação eficiente na Internet exige que cada computador ou dispositivo conectado tenha um endereço único e compatível com o *Internet Protocol* (IP). O sistema de endereçamento utilizado tem um profundo impacto na administração, gestão e escalabilidade da rede. Esta aula explora detalhadamente o conceito de endereçamento IP, a sua estrutura hierárquica e o impacto prático desta abordagem no funcionamento das redes modernas.

3.2 Endereços de Rede

Para que a comunicação entre dispositivos seja possível, é essencial que cada interface de rede tenha um endereço próprio e único. Cada interface é o ponto de conexão entre o dispositivo e a rede, sendo identificado por um endereço específico atribuído de acordo com um esquema globalmente organizado.

3.3 Requisitos para Endereçamento

Os endereços IP devem atender a alguns requisitos fundamentais:

- Serem globalmente únicos;
- Evitar atribuição manual extensiva e centralizada;
- Facilitar a implementação dos comutadores de pacotes;
- Manter tabelas de encaminhamento eficientes e compactas.

3.3.1 Soluções Iniciais e Limitações

Uma solução aparentemente simples seria a geração aleatória de endereços com um grande número de bits, semelhante às chaves criptográficas. No entanto, essa abordagem, embora descentralizada, levaria à criação de tabelas de encaminhamento enormes, causando sérios problemas de escalabilidade e gestão.

3.3.2 Hierarquia de Endereços

Para resolver os problemas mencionados, a utilização de estruturas hierárquicas torna-se uma solução eficiente e eficaz. Esta abordagem já é aplicada em diferentes áreas, como nos nomes DNS (e.g. www.di.uevora.pt) e nos endereços MAC atribuídos por fabricantes.

3.4 Estrutura dos Endereços IP

Um endereço IPv4 é constituído por 32 bits, geralmente representado na forma decimal pontuada (*dotted-quad notation*), como 193.136.216.21. Esta estrutura pode parecer arbitrária aos utilizadores, mas segue uma lógica hierárquica fundamental para o funcionamento da rede.

3 246 947 883 - Notação decimal

110000001100010000111111000101011 - Notação binária

193.136.126.43 - Dotted-quad notation

11000001	10001000	01111110	00101011
----------	----------	----------	----------

3.5 Hierarquia nos Endereços IP

Para os utilizadores (e computadores), os endereços IP parecem não ter qualquer estrutura, mas os encaminhadores de rede usam a informação contida nessa estrutura para encaminhar os pacotes.

A hierarquia nos endereços IP é crucial. Cada endereço é dividido em duas partes principais:

- **Prefixo de rede (network prefix)**, que identifica a rede;
- **Identificador do host (host identifier)**, que especifica um dispositivo na rede.

Por exemplo, o endereço 193.136.126.0/24 indica que 24 bits são utilizados para identificar a rede (*prefixo de rede*) e os 8 bits restantes para identificar os dispositivos específicos dentro dessa rede.

Para indicar claramente esta divisão entre rede e host, podemos usar máscaras de rede ou notação CIDR.

- Uma máscara como 255.255.255.0 mostra que os primeiros 24 bits são reservados para a rede, enquanto os restantes são reservados para o *número do host*.
- A notação CIDR (Classless Inter-Domain Routing) indica quantos bits fazem parte do prefixo (e.g. 193.136.120.0/21). A notação CIDR simplifica significativamente a gestão dos endereços ao agregar múltiplos endereços IP numa única representação compacta, representando efectivamente uma rede ou sub-rede.

3.6 Exemplos Práticos

Organizações como a FCCN ou a UEvora têm blocos de endereços IP agregados, o que simplifica a administração e permite uma fácil identificação e gestão nas tabelas de encaminhamento dos routers.

- A FCCN tem o bloco 193.136.0.0/15
Corresponde aos blocos 193.136.0.0/16 e 193.137.0.0/16
- A UEvora tem o bloco 193.136.216.0/22 (i.e., 4 blocos /24)
Corresponde aos blocos 193.136.216.0/24 até 193.136.219.0/24

3.7 Recursividade das Redes

O conceito de rede IP é inerentemente recursivo. Cada sub-rede está incluída numa rede maior, formando uma estrutura hierárquica que permite um encaminhamento eficiente dos pacotes de dados (figura 3.1).

3.8 Distribuição e Atribuição dos Endereços

A atribuição dos prefixos IP segue uma hierarquia rigorosa que começa na ICANN (Internet Corporation for Assigned Names and Numbers), passando pelas RIRs (Regional Internet Registries), depois pelos ISPs e, por último, pelas redes locais dos clientes (figura 3.2).

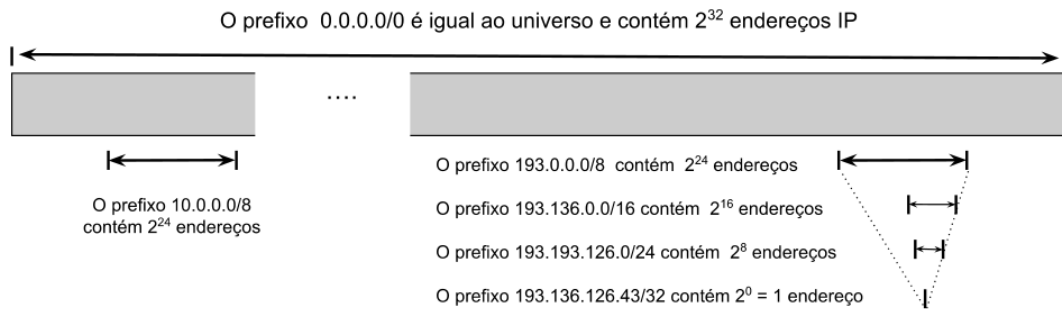


Figura 3.1: Hierarquia de redes IP

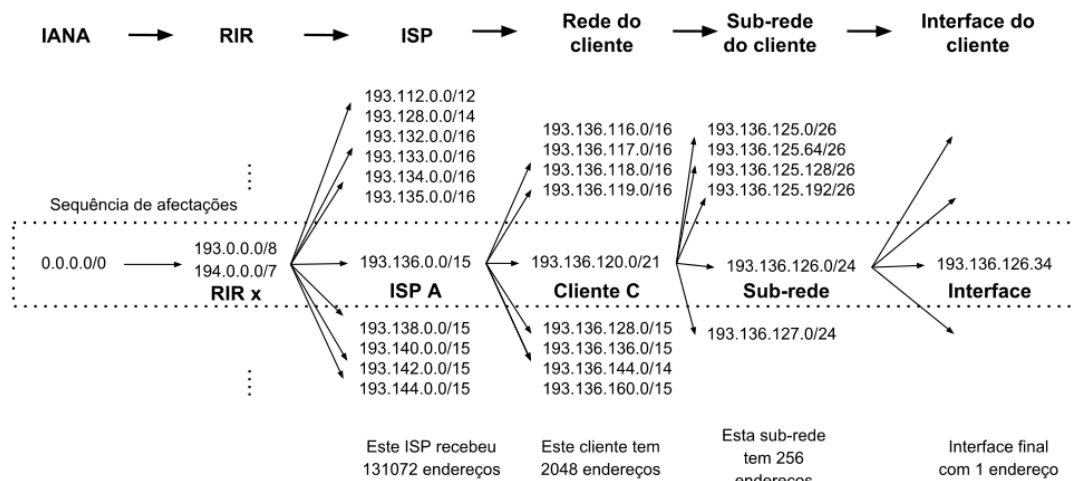


Figura 3.2: Atribuição dos endereços IP

Existem bases de dados públicas que permitem consultar os detentores de determinados blocos de endereços IP, através de comandos como `whois` em sistemas UNIX ou serviços online especializados.

```
$ whois -h whois.ripe.net 193.136.216.21
% This is the RIPE Database query service.
% The objects are in RPSL format.

% Information related to '193.136.216.0 - 193.136.219.255'

inetnum:        193.136.216.0 - 193.136.219.255
netname:        UE-PT-1
descr:          Universidade de Evora
descr:          Largo dos Colegiais
descr:          7000 Evora
country:        PT
geoloc:         38.572472 -7.904634
admin-c:        RCUD5-RIPE
tech-c:         RCUD5-RIPE
status:         ASSIGNED PA
org:            ORG-UDE1-RIPE
```

remarks: SERVIP-UEVORA

3.9 Funcionamento do Encaminhamento

Os comutadores ou routers usam tabelas de encaminhamento que relacionam prefixos IP às interfaces apropriadas para encaminhar os pacotes. O método conhecido como *Longest Prefix Matching* assegura que o pacote segue pela rota específica mais adequada.

A agregação dos prefixos IP é essencial para manter as tabelas de encaminhamento geríveis. Através da agregação de prefixos, redes grandes podem ser representadas por entradas únicas, reduzindo significativamente o tamanho das tabelas nos routers centrais da Internet.

3.10 Hierarquia e Limitações na Prática

Embora a estrutura hierárquica seja ideal para escalabilidade, situações reais, como multi-homing, em que uma rede está conectada a múltiplos fornecedores, podem desafiar esta abordagem. Nestes casos, são necessárias múltiplas entradas para representar a mesma rede, reduzindo parcialmente a eficácia da agregação.

3.11 Gamas de Endereços IPv4 Reservados

Existem vários blocos de endereços IPv4 que são reservados para usos específicos, não sendo considerados endereços *públicos*:

3.11.1 Redes Privadas (RFC 1918)

Gama de Endereços	Máscara Padrão	Hosts Disponíveis	Classe
10.0.0.0 – 10.255.255.255	255.0.0.0 (/8)	16.777.214	A
172.16.0.0 – 172.31.255.255	255.240.0.0 (/12)	1.048.574	B
192.168.0.0 – 192.168.255.255	255.255.0.0 (/16)	65.534	C

Tabela 3.1: Endereços IPv4 reservados para redes privadas.

3.11.2 Endereços de Loopback (RFC 1122)

A gama 127.0.0.0/8 é utilizada para comunicação interna do próprio dispositivo. O endereço mais conhecido é **127.0.0.1** (localhost).

3.11.3 Endereços Link-Local (RFC 3927)

A gama 169.254.0.0/16 é atribuída automaticamente quando um dispositivo não recebe um endereço IP via DHCP.

3.11.4 Endereços de Broadcast e Rede

- **Endereço de Rede:** Identifica uma rede (exemplo: 192.168.1.0/24). - **Endereço de Broadcast:** Envia pacotes para todos os hosts da rede (exemplo: 192.168.1.255/24).

3.11.5 Endereços Multicast (RFC 5771)

Os endereços entre 224.0.0.0 e 239.255.255.255 são usados para transmissão multicast.

3.11.6 Endereços para Documentação (RFC 5737)

Os seguintes blocos são reservados para documentação e exemplos:

- 192.0.2.0/24
- 198.51.100.0/24
- 203.0.113.0/24

3.11.7 Endereços Reservados para Uso Futuro

Os endereços 240.0.0.0/4 estão reservados para possível uso futuro.

3.12 Conclusão

Os endereços IP são o alicerce fundamental do funcionamento da Internet moderna. A sua estrutura hierárquica, embora simples na concepção, é crítica para a administração eficiente e escalável das redes globais, permitindo uma gestão eficaz, encaminhamento eficiente e flexibilidade suficiente para lidar com situações complexas do mundo real.

4. O Protocolo IP [10.03.2025]

4.1 Introdução e Contextualização

A Internet é uma rede global composta por uma diversidade de redes heterogêneas. Para garantir a comunicação entre sistemas com diferentes tecnologias, é fundamental que haja um conjunto de regras e convenções comuns. O protocolo IP (Internet Protocol) foi desenvolvido exatamente para este propósito, possibilitando o endereçamento e o encaminhamento de pacotes através de múltiplas redes, mesmo que estas utilizem infraestruturas físicas e lógicas distintas.

O IP define o formato dos pacotes, os campos necessários para a identificação dos nós e as regras de encaminhamento que permitem que os pacotes encontrem o caminho mais eficiente até ao seu destino. Esta funcionalidade é essencial para que a Internet se mantenha escalável e robusta, permitindo a comunicação entre milhões de dispositivos espalhados pelo mundo.

Esta aula apresenta os fundamentos, mecanismos e desafios associados ao protocolo IP, que é o elemento central no encaminhamento de pacotes na Internet.

4.2 Posição do Protocolo IP na Stack TCP/IP

A arquitetura TCP/IP organiza a comunicação em camadas, cada uma com funções específicas:

- **Camada de Aplicação:** Protocolos como HTTP, FTP, DNS, entre outros, possibilitam a interação entre programas e serviços.
- **Camada de Transporte:** Protocolos como TCP e UDP asseguram a entrega correta ou rápida dos dados, conforme as necessidades da aplicação.
- **Camada de Rede:** Aqui temos o protocolo IP, que define o endereçamento, a estrutura dos pacotes e os mecanismos de encaminhamento.
- **Camada Física/Data-Link:** Protocolos como Ethernet e Wi-Fi definem como os dispositivos acedem ao meio físico e encapsulam os pacotes em frames para a transmissão local.

Nesta estrutura, o IP atua como interface entre redes, permitindo que um pacote originado numa rede local seja transmitido através de diversas tecnologias até atingir o destino final. Este mecanismo de interligação entre redes é o que permite à Internet funcionar de forma coesa.

4.3 Formato de um Pacote IP (IPv4)

O pacote IP é composto por um cabeçalho e um payload (dados). O cabeçalho do IPv4, sem opções, tem um tamanho fixo de 20 bytes e contém diversos campos essenciais:

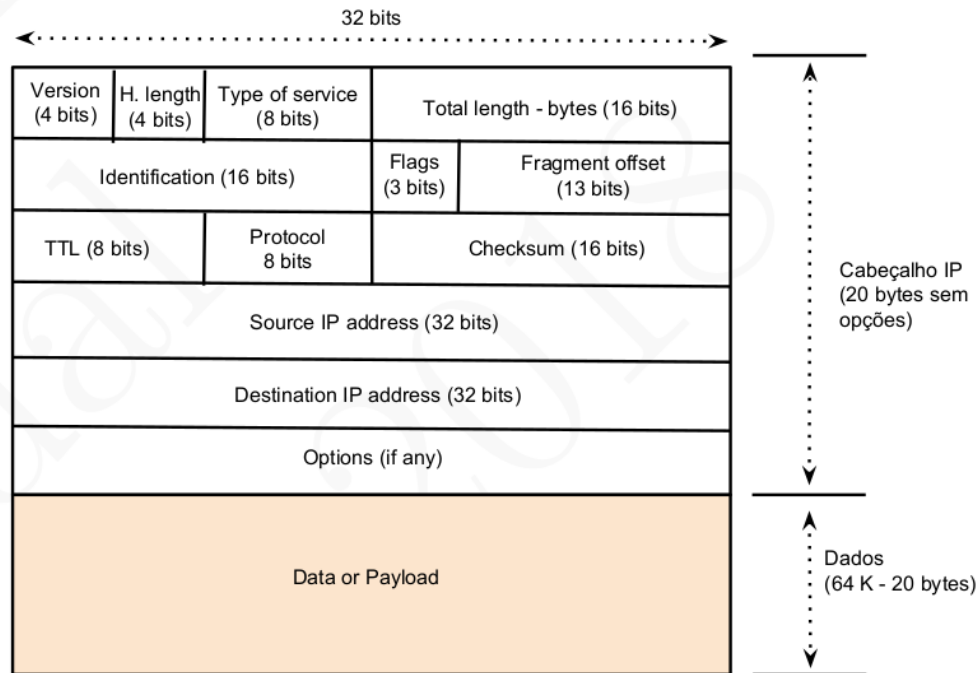


Figura 4.1: Formato de um pacote IPv4

4.3.1 Campos Básicos do Cabeçalho IPv4

- **Versão (4 bits):** Indica a versão do protocolo, sendo o valor 4 para IPv4. Este campo é fundamental para que o receptor saiba como interpretar o restante do cabeçalho.
- **Header Length (4 bits):** Define o número de palavras de 32 bits que compõem o cabeçalho. Normalmente, este valor é 5, correspondendo a 20 bytes.
- **Tipo de Serviço (8 bits):** Define a prioridade do pacote, permitindo classificações baseadas em diferentes níveis de serviço. Embora muitas vezes este campo seja ignorado pelos routers, pode ser útil na implementação de políticas de QoS.
- **Total Length (16 bits):** Indica o tamanho total do pacote, incluindo cabeçalho e dados. Embora o tamanho máximo seja 64K bytes, na prática os pacotes são frequentemente menores devido a restrições impostas pelo meio de transmissão (camada física).

4.3.2 Campos de Fragmentação, TTL, Protocolo e Checksum

- **Fragmentação:** Se o pacote exceder o tamanho máximo permitido pela camada física, poderá ser fragmentado. O cabeçalho inclui campos para identificação, flags e offset que permitem ao destino reconstruir o pacote original a partir dos seus fragmentos.
- **Time-to-Live (TTL, 8 bits):** Este campo define o número máximo de saltos (hops, routers) que o pacote pode atravessar. A cada passagem por um router, o valor do TTL é decrementado; se atingir zero, o pacote é descartado, evitando ciclos infinitos na rede.
- **Protocolo (8 bits):** Identifica o protocolo que está no payload, como TCP (6) ou UDP (17), permitindo a desmultiplexagem correta no nível de transporte.
- **Header Checksum (16 bits):** É usado para verificar a integridade do cabeçalho. O cálculo é feito através da soma de todas as palavras de 16 bits do cabeçalho; se houver discrepância no valor recalculado pelo receptor, o pacote é descartado.

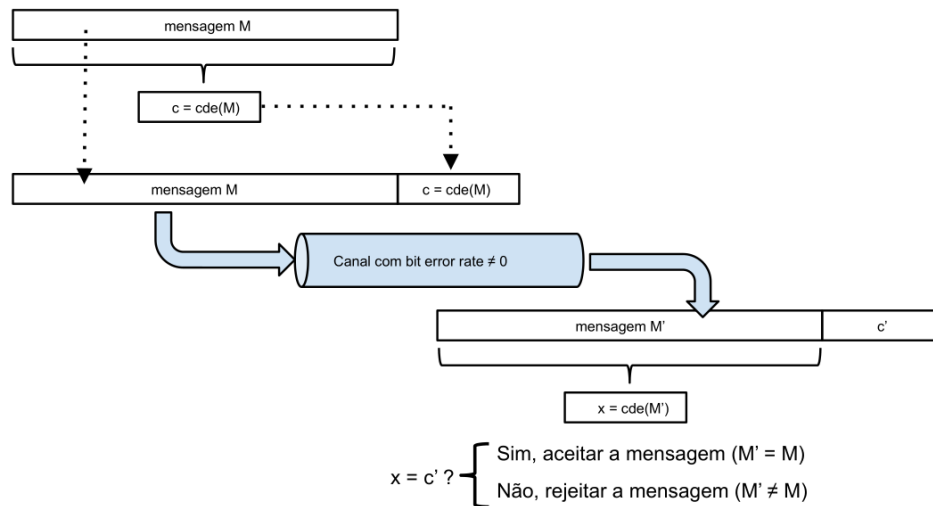


Figura 4.2: Checksum IP: Função CDE (Código de Detecção de Erros)

4.3.3 Opções

Embora o cabeçalho padrão tenha 20 bytes, o campo de opções pode ser incluído para funcionalidades adicionais, como a especificação de rotas (source route) ou a gravação de rota (record route). Contudo, na prática, o uso de opções é raro devido à complexidade e ao impacto no desempenho.

4.4 Encaminhamento de Pacotes: Direto, Indireto e por Omissão

O encaminhamento de pacotes é um dos processos centrais no funcionamento do IP. Cada nó na rede (computador ou router) utiliza tabelas de encaminhamento¹ para decidir o melhor caminho para um pacote, com base no endereço IP de destino.

4.4.1 Tabelas de Encaminhamento

Cada dispositivo mantém uma tabela onde se encontram entradas do tipo:

- Endereços ou prefixos que representam as redes diretamente ligadas.
- Informações sobre gateways (routers vizinhos) para destinos que não estão na rede local.
- Uma rota por omissão, normalmente representada pelo prefixo 0.0.0.0/0, que indica o caminho a seguir quando não há uma correspondência mais específica.

Para ilustrar os diferentes tipos de encaminhamento, usaremos uma rede exemplo 80.80.0.0/22, subdividida nas redes 80.80.1.0/24 (ethernet), 80.80.2.0/24 (wi-fi) e 80.80.3.0/30 (Point-to-Point). A figura 4.3 mostra um esquema da rede. Podemos ver a tabela de encaminhamento do computador com o endereço 80.80.1.3 na tabela 4.1.

Tabela 4.1: Tabela de encaminhamento (forwarding table) do computador com endereço 80.80.1.3

Rede ou Prefixo	Tipo de encaminhamento	Gateway	Interface	Métrica
80.80.1.3/32	Endereço local	80.80.1.3	eth0	0
80.80.1.0/24	Direto	80.80.1.3	eth0	0
80.80.2.0/24	Indireto	80.80.1.253	eth0	1
80.80.3.0/30	Indireto	80.80.1.254	eth0	1
0.0.0.0/0	Indireto	80.80.1.254	eth0	1

¹Podemos consultar a tabela de encaminhamento do nosso computador através dos comandos 'route' ou 'ip route'.

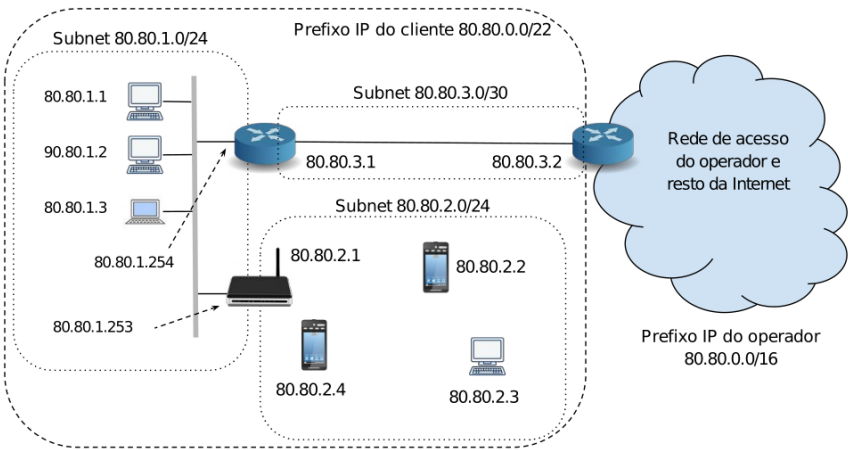


Figura 4.3: Exemplo de rede.

4.4.2 Encaminhamento Direto

O encaminhamento direto ocorre quando o endereço de destino pertence à mesma sub-rede à qual o dispositivo está conectado. Nessa situação, não é necessário o encaminhamento por uma gateway, e o pacote pode ser enviado diretamente através da interface local. O processo envolve, muitas vezes, o uso do protocolo ARP para determinar o endereço MAC correspondente ao endereço IP de destino (figura 4.4).

A tabela 4.2 mostra as entradas que podem ser usadas quando o computador com endereço IP 80.80.1.3 pretende enviar um pacote para o computador 80.80.1.1, prevalecendo a que tem o maior prefixo (*Longest-prefix matching*).

Tabela 4.2: Matching entries para o computador 80.80.1.1

Rede ou Prefixo	Tipo de encaminhamento	Gateway	Interface	Métrica
80.80.1.3/32	Endereço local	80.80.1.3	eth0	0
80.80.1.0/24	Direto	80.80.1.3	eth0	0
80.80.2.0/24	Indireto	80.80.1.253	eth0	1
80.80.3.0/30	Indireto	80.80.1.254	eth0	1
0.0.0.0/0	Indireto	80.80.1.254	eth0	1

4.4.3 Encaminhamento Indireto

Quando o destino não se encontra na mesma sub-rede, o pacote deve ser encaminhado para um router que o aproxime do destino. Este processo é denominado encaminhamento indireto. A decisão é tomada através da consulta à tabela de encaminhamento, que indica qual a gateway mais adequado para a rede de destino (figura 4.5).

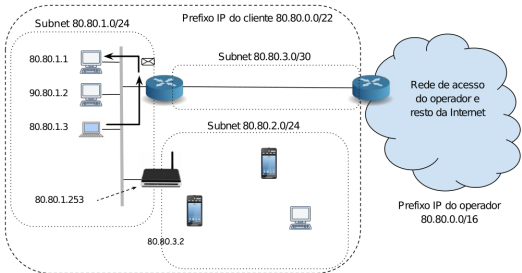


Figura 4.4: Pacote para 80.80.1.1

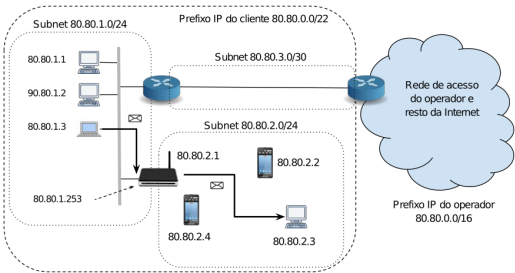


Figura 4.5: Pacote para 80.80.2.3

Podemos ver as linhas da tabela de encaminhamento na tabela 4.3.

Tabela 4.3: *Matching entries* para o computador 80.80.2.3

Rede ou Prefixo	Tipo de encaminhamento	Gateway	Interface	Métrica
80.80.1.3/32	Endereço local	80.80.1.3	eth0	0
80.80.1.0/24	Direto	80.80.1.3	eth0	0
80.80.2.0/24	Indireto	80.80.1.253	eth0	1
80.80.3.0/30	Indireto	80.80.1.254	eth0	1
0.0.0.0/0	Indireto	80.80.1.254	eth0	1

4.4.4 Encaminhamento por Omissão

Se nenhuma entrada na tabela de encaminhamento corresponder especificamente ao endereço de destino, utiliza-se a rota por omissão. Esta rota, identificada pelo prefixo 0.0.0.0/0, garante que o pacote seja enviado para um router que tenha conhecimento de como alcançar redes remotas. Este mecanismo assegura que, mesmo sem uma correspondência exata, o pacote tem melhores hipóteses de ser entregue, através da delegação para o *hop* seguinte (figura 4.6).

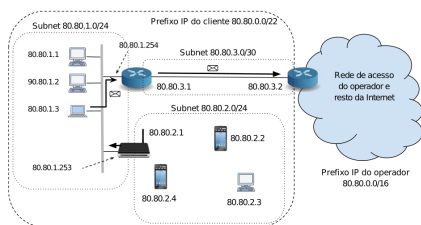


Tabela 4.4: *Matching entry* para o computador 193.136.216.21

Rede ou Prefixo	Tipo de encaminhamento	Gateway	Interface	Métrica
80.80.1.3/32	Endereço local	80.80.1.3	eth0	0
80.80.1.0/24	Direto	80.80.1.3	eth0	0
80.80.2.0/24	Indireto	80.80.1.253	eth0	1
80.80.3.0/30	Indireto	80.80.1.254	eth0	1
0.0.0.0/0	Indireto	80.80.1.254	eth0	1

Figura 4.6: Pacote para 193.136.216.21

4.4.5 Tratamento de Pacotes nos Routers

Ao receber um pacote, o router realiza os seguintes passos:

1. Verifica se o endereço de destino corresponde ao seu próprio endereço (caso em que o pacote é processado localmente).
2. Consulta a tabela de encaminhamento para identificar a melhor rota com base no *Longest-Prefix Matching*.
3. Se encontrar uma entrada que corresponda, encaminha o pacote através da interface indicada.
4. Se o pacote não corresponder a nenhuma entrada específica, utiliza a rota por omissão.
5. Caso o TTL do pacote atinja zero, o pacote é descartado e, frequentemente, é enviada uma mensagem ICMP ao remetente, indicando que o tempo (TTL) expirou.

4.5 Protocolos Complementares ao IP

Para assegurar a eficácia do encaminhamento e a correta atribuição de endereços, o protocolo IP é complementado por outros protocolos fundamentais: ARP, DHCP e ICMP.

4.5.1 ARP: Address Resolution Protocol

O ARP é utilizado para associar endereços IP a endereços físicos (MAC) numa rede local. Cada dispositivo mantém uma tabela ARP, onde são armazenados pares (IP, MAC). Quando um dispositivo precisa de enviar um pacote e o endereço MAC correspondente não está na sua tabela, envia uma mensagem de broadcast com o pedido "Who has IP address X?". O dispositivo que tem esse endereço responde com o seu endereço MAC, permitindo a atualização da tabela ARP e o envio do pacote.

4.5.2 DHCP: Dynamic Host Configuration Protocol

O DHCP automatiza a atribuição de parâmetros de rede (como endereço IP, máscara, gateway e servidores DNS) aos dispositivos que se ligam à rede. O processo típico envolve:

1. **DHCP Discover:** O cliente envia uma mensagem de broadcast solicitando parâmetros de configuração.
2. **DHCP Offer:** Um ou mais servidores DHCP respondem oferecendo configurações.
3. **DHCP Request:** O cliente escolhe uma oferta e solicita formalmente a atribuição.
4. **DHCP ACK:** O servidor confirma a atribuição, definindo o tempo de lease para a validade da configuração.

Esta automatização elimina a necessidade de configuração manual, reduzindo erros e conflitos de endereçamento.

4.5.3 ICMP: Internet Control Message Protocol

O ICMP complementa o IP ao fornecer mensagens de controlo e diagnóstico. Embora o IP seja um protocolo *best effort* sem garantias de entrega, o ICMP permite comunicar problemas ocorridos durante o encaminhamento, tais como:

- **Echo Request e Echo Reply:** Utilizados pelo programa `ping` para testar a conectividade e medir o tempo de resposta (RTT).
- **Destination Unreachable:** Indicam que um pacote não pôde ser entregue devido a problemas como falta de rota ou portas inativas.
- **Time Exceeded:** Enviadas quando o TTL de um pacote atinge zero, alertando sobre a existência de possíveis ciclos de encaminhamento.

Ferramentas como `traceroute` fazem uso das mensagens ICMP para mapear o caminho percorrido pelos pacotes até ao destino, permitindo identificar gargalos ou pontos de falha na rede.

4.6 Exemplos Práticos e Aplicações

Para consolidar os conceitos teóricos, é útil analisar alguns exemplos práticos que ilustram o funcionamento dos mecanismos descritos.

4.6.1 Utilização do TTL e a Ferramenta Traceroute

O campo *Time-to-Live* (TTL) é crucial para evitar ciclos infinitos na rede. Cada router decrementa o valor do TTL em 1. Se o valor chegar a zero, o pacote é descartado e, geralmente, é enviada uma mensagem ICMP informando o emissor.

A ferramenta `traceroute` explora esta funcionalidade enviando pacotes com valores crescentes de TTL. Cada router que recebe um pacote com TTL expirado responde com uma mensagem "Time Exceeded". Assim, é possível mapear o percurso dos pacotes até ao destino e identificar a localização de eventuais problemas ou atrasos.

4.6.2 O Programa Ping para Diagnóstico de Redes

O `ping` é uma ferramenta amplamente utilizada para testar a conectividade entre dois dispositivos. Funciona através do envio de pacotes ICMP `Echo Request` e da receção de `Echo Reply`. Cada pacote contém um número de sequência e um registo do tempo de envio. A diferença entre o tempo de envio e o tempo de receção permite calcular o *Round Trip Time* (RTT), que é uma medida de latência entre os nós.

4.7 IPv6: A Evolução do Protocolo IP

Com o crescimento exponencial de dispositivos conectados à Internet, o espaço de endereçamento do IPv4 revelou-se insuficiente. Como resposta, foi desenvolvido o IPv6, que traz melhorias não só no espaço de endereçamento, mas também em outros aspetos do protocolo.

4.7.1 Principais Características do IPv6

- **Endereços de 128 bits:** Permitem um número quase infinito de endereços, solucionando o problema do esgotamento dos endereços IPv4.
- **Cabeçalho Simplificado:** Embora o cabeçalho do IPv6 tenha um tamanho fixo de 40 bytes, elimina campos considerados redundantes no IPv4, como o *Header Checksum*, delegando a deteção de erros a níveis inferiores.
- **Melhoria no Roteamento:** O IPv6 permite uma hierarquia de endereços mais eficiente, facilitando a agregação de rotas e a redução do tamanho das tabelas de encaminhamento.
- **Qualidade de Serviço (QoS):** Introduz o campo *Flow Label*, que pode ser utilizado para identificar e tratar fluxos de dados com necessidades específicas, embora a sua implementação prática ainda seja limitada.

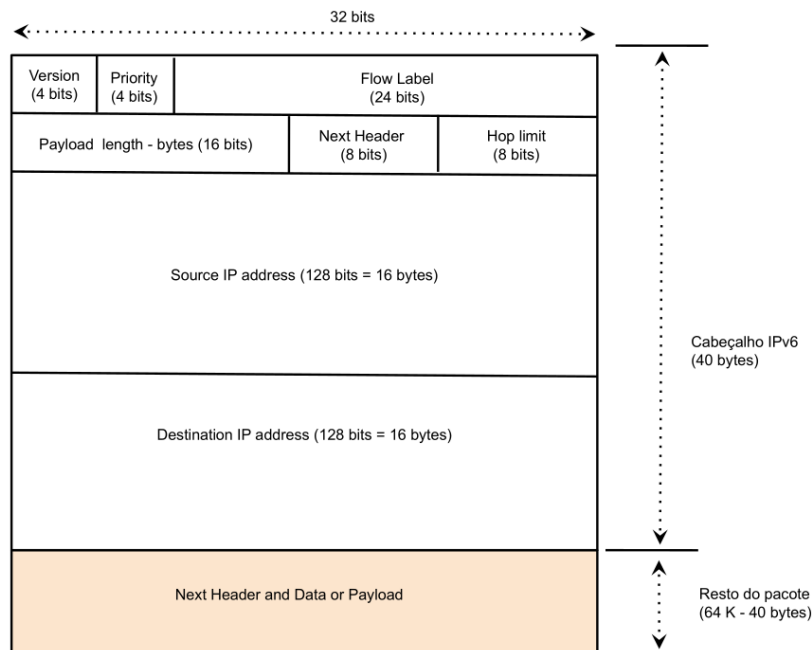


Figura 4.7: Formato de um pacote IPv6

4.7.2 Formato do Pacote IPv6

O cabeçalho do IPv6 inclui os seguintes campos:

- **Versão (4 bits):** Valor fixo 6, indicando a utilização do IPv6.
- **Flow Label (24 bits):** Permite a identificação de fluxos de dados para tratamento diferenciado.
- **Payload Length (16 bits):** Indica o tamanho do payload (dados) que segue o cabeçalho.
- **Next Header (8 bits):** Define o protocolo do próximo nível, equivalente ao campo *Protocolo* no IPv4.
- **Hop Limit (8 bits):** Funciona de forma análoga ao TTL do IPv4, limitando o número de saltos que o pacote pode efetuar.
- **Endereço IP de Origem e Destino (128 bits cada):** Permitem uma identificação globalmente única dos

computadores.

4.7.3 Transição e Coexistência

A transição do IPv4 para o IPv6 tem sido gradual. Enquanto muitas redes já implementaram o IPv6, a coexistência dos dois protocolos é comum. Existem mecanismos, como o *Dual Stack* e técnicas de *tunneling*, que permitem a comunicação entre dispositivos que utilizam protocolos IP de versões diferentes, assegurando uma migração suave e contínua.

4.8 Network Address Translation (NAT)

Devido à limitação de endereços no IPv4, o NAT (Network Address Translation) tornou-se uma solução prática para permitir que múltiplos dispositivos numa rede privada partilhem um único endereço IP público. No entanto, o NAT introduz certas complexidades que merecem análise.

4.8.1 Funcionamento do NAT

O NAT opera alterando os endereços IP nos cabeçalhos dos pacotes à medida que estes atravessam o router:

- **Tradução de Endereços:** Os endereços privados (por exemplo, no intervalo 10.0.0.0/8, 172.16.0.0/12 ou 192.168.0.0/16) são substituídos por um endereço IP público atribuído pelo ISP.
- **Mapeamento de Portas:** Para distinguir as diferentes sessões de comunicação, o NAT também altera os números de porta. Assim, cada conexão estabelecida entre um computador interno e outro externo é identificada de forma única.
- **Manutenção de Tabelas:** O router NAT mantém uma tabela que associa os endereços e portas originais aos novos valores utilizados para a comunicação com a Internet.

4.8.2 Vantagens e Desvantagens do NAT

Vantagens:

- Permite a utilização de um único endereço IP público para vários dispositivos, o que é crucial face à escassez de endereços IPv4.
- Aumenta a segurança, uma vez que os endereços internos ficam ocultos para o exterior.
- Facilita a portabilidade da rede interna, permitindo mudanças de ISP sem necessidade de reconfiguração dos endereços internos (que se mantêm).

Desvantagens:

- Pode complicar a comunicação de aplicações que necessitam de estabelecer ligações diretas, como alguns jogos online ou aplicações de VoIP.
- Introduce uma camada adicional de processamento, que pode aumentar a latência em ambientes com elevado tráfego.
- Torna a configuração de serviços acedidos diretamente a partir da Internet mais complexa, recorrendo muitas vezes a técnicas como o *port forwarding*.

4.8.3 Port Forwarding e Relaying

Para permitir o acesso a servidores “escondidos” numa rede privada, são utilizadas técnicas como o *port forwarding*. Esta técnica redireciona tráfego que chega a uma porta específica do endereço IP público para o servidor interno designado. Outra solução é o *relaying*, onde um servidor intermédio atua como ponte para a comunicação entre dispositivos internos e externos.

4.9 Gamas de Endereços IPv4 Privados

Para preservar o espaço de endereçamento público, foram definidas gamas de endereços privados que podem ser utilizados internamente sem serem roteáveis na Internet. Repetimos aqui as principais gamas de endereços privados:

- **10.0.0.0/8:** Abrange cerca de 16 milhões de endereços, ideal para redes muito grandes.
- **172.16.0.0/12:** Disponibiliza aproximadamente 1 milhão de endereços, adequada para redes de dimensão média.
- **192.168.0.0/16:** Com cerca de 65 mil endereços, é a mais frequentemente utilizada em redes domésticas e pequenas empresas.

Estes intervalos são reservados para uso interno e, quando combinados com o NAT, permitem que redes privadas comuniquem com a Internet sem a necessidade de ter endereços públicos para cada dispositivo.

4.10 Aplicações e Impactos Práticos do Protocolo IP

A compreensão do protocolo IP e dos seus mecanismos de encaminhamento é essencial para o desenho e a manutenção de redes de computadores. Abaixo, destacam-se algumas aplicações práticas e implicações deste conhecimento:

4.10.1 Diagnóstico e Monitorização de Redes

Ferramentas como `ping` e `traceroute` são amplamente utilizadas para diagnosticar problemas de conectividade. Ao analisar os tempos de resposta e o percurso dos pacotes, é possível identificar:

- Gargalos e atrasos na rede.
- Routers ou segmentos com problemas de encaminhamento.
- Configurações incorretas ou mal configuradas em tabelas de encaminhamento.

4.10.2 Planeamento e Otimização de Redes

O estudo aprofundado do IP permite aos engenheiros de redes:

- Planear a estrutura de endereçamento de uma rede, assegurando a adequada segmentação e alocação de subredes.
- Otimizar as tabelas de encaminhamento para reduzir a latência e melhorar a eficiência na transmissão de dados.
- Implementar mecanismos de segurança, como a segregação de redes e a utilização de NAT, para proteger os dispositivos internos.

4.10.3 Evolução para Novas Tecnologias

Com o aumento exponencial de dispositivos conectados e a expansão das aplicações móveis e da Internet das Coisas (IoT), a transição para o IPv6 é uma realidade inevitável. Esta evolução não só aumenta o espaço de endereçamento, como também introduz melhorias no roteamento e na segurança, adaptando a Internet às necessidades futuras.

4.11 Desafios e Perspetivas Futuras

Embora o protocolo IP seja um alicerce fundamental da comunicação na Internet, o seu funcionamento enfrenta vários desafios, entre os quais se destacam:

4.11.1 Escassez de Endereços IPv4

O esgotamento do espaço de endereços IPv4 levou ao desenvolvimento de técnicas como o NAT e, em última instância, à migração para o IPv6. Contudo, a coexistência dos dois protocolos durante o período de transição implica desafios técnicos e operacionais que continuam a ser objeto de estudo.

4.11.2 Segurança e Confiabilidade

O IP, por si só, não fornece mecanismos robustos de segurança. Protocolos complementares e técnicas adicionais são necessários para mitigar riscos, como:

- Ataques de negação de serviço (DoS).
- Exploração de vulnerabilidades nos processos de encaminhamento.
- Intercepção e manipulação de pacotes (Man-in-the-Middle).

4.11.3 Qualidade de Serviço (QoS) e Prioritização de Tráfego

Num ambiente com um volume elevado de tráfego e aplicações com diferentes requisitos (por exemplo, streaming de vídeo versus comunicação em tempo real), a priorização de pacotes torna-se crucial. Embora o campo *Tipo de Serviço* no IPv4 e o *Flow Label* no IPv6 ofereçam algum suporte à qualidade de serviço, a implementação prática destas funcionalidades ainda é um desafio.

4.11.4 Interoperabilidade e Transição para o IPv6

A coexistência de IPv4 e IPv6 durante o período de transição impõe a necessidade de mecanismos que garantam a interoperabilidade entre os dois protocolos. Soluções como *Dual Stack* e *tunneling* são essenciais, mas requerem um planeamento cuidadoso e a adaptação de equipamentos e softwares.

4.12 Considerações Finais e Reflexões sobre o Protocolo IP

O protocolo IP define a espinha dorsal da comunicação na Internet. Ao definir o formato dos pacotes, os mecanismos de encaminhamento e a forma como os dispositivos interagem, o IP possibilita a interligação de redes heterogêneas e a construção de uma infraestrutura global de comunicação.

Para os alunos de redes de computadores, o estudo deste protocolo é fundamental, não só pela sua importância histórica e técnica, mas também pelas perspectivas futuras que ele oferece. O domínio dos conceitos associados ao IP permite compreender melhor os desafios atuais e participar na evolução contínua das tecnologias de rede.

Ao integrar o conhecimento sobre ARP, DHCP, ICMP, NAT e a transição para o IPv6, os futuros profissionais estarão melhor preparados para enfrentar os desafios de uma Internet em constante expansão, caracterizada por uma crescente procura por conectividade, segurança e eficiência.

4.13 Para Lembrar

- **Posição na Stack TCP/IP:** O IP é o elo que permite a comunicação entre redes distintas, interligando aplicações, transporte e meios físicos.
- **Estrutura do Pacote IPv4:** Estudámos os campos essenciais do cabeçalho IPv4, incluindo endereçamento, fragmentação, TTL e checksum.
- **Encaminhamento de Pacotes:** Analisámos os diferentes métodos de encaminhamento – direto, indireto e por omissão – e a forma como os routers utilizam as tabelas de encaminhamento para direcionar os pacotes.

- **Protocolos Complementares:** Destacámos o papel dos protocolos ARP, DHCP e ICMP na manutenção da comunicação e na resolução de problemas de rede.
- **Evolução para o IPv6:** Discutimos as melhorias do IPv6, tanto no espaço de endereçamento quanto na eficiência do roteamento e na potencial implementação de QoS.
- **Network Address Translation (NAT):** Explicámos o funcionamento do NAT, as vantagens e desvantagens associadas e as técnicas de port forwarding e relaying.
- **Desafios e Perspetivas Futuras:** Refletimos sobre os principais desafios, como a escassez de endereços, questões de segurança e a necessidade de interoperabilidade durante a transição para o IPv6.

Esta aula pretendeu fornecer uma visão abrangente dos mecanismos que sustentam a comunicação na Internet, enfatizando a importância do protocolo IP como elemento central na construção e manutenção de redes de computadores.

4.14 Conclusão

O estudo do protocolo IP revela não só a complexidade intrínseca à comunicação em redes de computadores, mas também a elegância de um sistema que, através de regras bem definidas, permite a interligação de milhões de dispositivos num ambiente global. Compreender estes conceitos é indispensável para o desenvolvimento e a manutenção de infraestruturas de rede modernas, capazes de responder às exigências de um mundo cada vez mais conectado.